CloneScope: Complete Tokenization Learning Manual

A beginner-friendly, code-grounded guide to preprocessing, lexical analysis, tests, architecture, and viva defense

Prepared for the SPL-1 project of Raian Huq (BSSE1708)

27 October 2026

Contents

How to use this manual **4**

1. What does this stage of CloneScope do? **4**
1.1 Explanation for a nonprogrammer . . . 4
1.2 Explanation for a software engineering classmate . . . 4
1.3 Explanation for a professor . . . 4
1.4 Input, transformation, output . . . 5
1.5 Architecture map . . . 5

2. Essential vocabulary **6**
2.1 Source, character, lexeme, token . . . 6
2.2 Why code comparison needs tokens . . . 6
2.3 Taxonomy in this project . . . 6

3. C++17 concepts needed to read this implementation **7**
3.1 Header vs implementation file . . . 7
3.2 `#include` and `#pragma once` . . . 7
3.3 `enum class` represents a controlled category . . . 7
3.4 `struct Token` combines related data . . . 8
3.5 `std::vector<Token>` stores an ordered sequence . . . 8
3.6 `const std::string& source` . . . 8
3.7 `std::unordered_set` and membership checks . . . 8
3.8 Important control-flow syntax . . . 9
3.9 Character classification and safe conversion . . . 9
3.10 `std::string::npos` . . . 9

4. Project files and responsibilities **9**
4.1 Minimal `main()` is intentional . . . 10
4.2 The application interface . . . 10

5. File reading and folder scanning **10**
5.1 Getting the path . . . 10
5.2 Selecting folder mode . . . 11
5.3 Opening and reading an entire file . . . 11
5.4 The processing hand-off . . . 11

6. Preprocessor state machine: concept first **12**
6.1 Why a simple search-and-replace is incorrect . . . 12
6.2 The five states . . . 12
6.3 State transition sketch . . . 12
6.4 Why mask comments instead of deleting bytes? . . . 13

7. Preprocessor implementation: line-by-line reasoning **13**
7.1 Function signature and variables . . . 13
7.2 Detecting a `//` opener . . . 13

7.3 Detecting `/*` and `*/` . . . 14
7.4 Entering quoted regions . . . 14
7.5 Ending line comments . . . 14
7.6 Protecting escape pairs inside quotes . . . 15
7.7 Malformed ordinary-literal recovery . . . 15
7.8 What this preprocessor does not do . . . 15

8. Tokenizer architecture: the main scanning loop 15
8.1 Interface and initial state . . . 15
8.2 Rule order and why it matters . . . 16
8.3 Keyword recognition table . . . 16

9. Scanning identifiers and keywords 16
9.1 Statement-by-statement commentary . . . 17
9.2 Worked trace: `int total_2;` . . . 17

10. Scanning decimal numbers 18
10.1 Why there are two digit loops . . . 18
10.2 What works vs what doesn't . . . 18

11. Quoted literals and escape sequences 19
11.1 String-literal rule . . . 19
11.2 Character-literal rule . . . 20
11.3 Three different backslash cases . . . 20
11.4 Escaped quote example . . . 20
11.5 Malformed ordinary literals . . . 20

12. Punctuation, operators, and the longest supported match 21
12.1 Structural punctuation . . . 21
12.2 One-character operators . . . 21
12.3 Two-character operators . . . 21
12.4 Longest *supported* match . . . 21
12.5 Worked operator trace . . . 22

13. Unknown tokens, whitespace, and positions 22
13.1 Why unknowns must not vanish . . . 22
13.2 Newline vs other whitespace . . . 22
13.3 Worked example including unknown . . . 23

14. Full worked source trace 23
14.1 Expected tokens by line . . . 23
14.2 Selected index-to-coordinate checks . . . 23
14.3 Pencil-and-paper tracing method . . . 24

15. Automated tests: what was actually verified 24
15.1 Independent preprocessor smoke test . . . 24
15.2 Independent tokenizer smoke test . . . 24
15.3 Automated file pipeline smoke test . . . 25
15.4 Why assert *both* count and content? . . . 25
15.5 Red-green regression example . . . 25
15.6 `assert` caveat . . . 25

16. Exact Windows PowerShell build and run reference 25
16.1 Preprocessor test . . . 25
16.2 Tokenizer test . . . 26
16.3 File pipeline test . . . 26
16.4 Complete application . . . 26
16.5 Why each `.cpp` belongs in a command . . . 26
16.6 Compiler switches explained . . . 26

17. Git workflow and verified milestones 26
17.1 Why work on a feature branch? . . . 26

17.2 Milestone commits observed during development . . . 27
17.3 Useful Git verification commands . . . 27
17.4 What a safe PR review checks . . . 27

18. Complexity analysis and design tradeoffs 27
18.1 Time complexity . . . 27
18.2 Why not regular expressions for everything? . . . 28
18.3 Why separate preprocessing from tokenization? . . . 28
18.4 Tradeoffs of fixed lexical subsets . . . 28

19. Explicit limitations and risks to disclose 28

20. Supervisor viva: conceptual questions and model answers 29

21. Supervisor viva: implementation, testing, architecture and ethics 30

22. Practice exercises and fully explained solutions 31
Exercise 1: Identify the token categories . . . 31
Exercise 2: Why is `node->value` three tokens? . . . 31
Exercise 3: Trace a physical newline . . . 32
Exercise 4: Comment masking and columns . . . 32
Exercise 5: Can the tokenizer recognize `.5` as one number? . . . 32
Exercise 6: Why would a missing quote change preprocessing? . . . 32
Exercise 7: Find the number of tokens . . . 32
Exercise 8: Explain a linker error . . . 32

23. Final pre-merge checklist 32

Appendix A. Reference interfaces (actual basic design) 33
A.1 `tokenizer.hpp` . . . 33
A.2 `preprocessor.hpp` . . . 33
A.3 `application.hpp` . . . 34

Appendix B. Tokenization quick-reference tables 34
B.1 Location update rules . . . 34
B.2 Supported single-/two-character operator declaration . . . 34

Appendix C. Further self-study: reading order 34

Purpose. This manual explains the *basic preprocessing and tokenization implementation actually built and tested* in CloneScope. It is intended to be understandable to a student encountering lexical analysis for the first time. It explains both the **what** and **why**, not just code to memorize.

Scope boundary. CloneScope's approved project encompasses explainable code-clone tagging and stylometric analysis, but **clone matching, stylometry, batch pairwise statistics and evidence reports are not implemented by the tokenizer milestone described here**. Tokenization produces source-code representations for later comparison; it does **not** determine plagiarism.

Source basis. Examples, functions, tests, and build commands below are based on the C++ code and execution screenshots reviewed during the CloneScope sessions through 27 October 2026. The manual is an educational reference, not a claim that the entire C/C++ language is implemented. A fresh Git checkout remains the source of truth for final repository content.

How to use this manual

If you are learning from zero, read Chapters 1-4 first, then study the preprocessor and tokenizer chapters, then trace the examples with pencil and paper. Before a supervisor meeting, study Chapters 14-17 and explain the pipeline without looking at the code. The reference appendices are for revisiting code signatures, commands, and tests.

Four learning passes:

1. **Big picture:** Describe what goes in, what each stage does, and what comes out.
2. **Code comprehension:** Explain each new construct such as `std::vector<Token>`, `std::size_t`, `substr`, and `continue`.
3. **Execution tracing:** Follow `index`, `line`, `column`, and preprocessor `state` across small examples.
4. **Verification:** Compile separate modules and explain why every assertion is necessary.

1. What does this stage of CloneScope do?

1.1 Explanation for a nonprogrammer

CloneScope currently reads C/C++ code, hides comments without moving the remaining code, and breaks the remaining text into labeled pieces called **tokens**. Each piece remembers where it came from. Later project stages can compare these token sequences and produce evidence for a human reviewer.

1.2 Explanation for a software engineering classmate

The basic pipeline is a command-line C++17 application with single-file and nonrecursive folder input. It reads each eligible `.c` / `.cpp` file, passes text through a comment-removing lexical state machine, passes the cleaned text through a rule-based tokenizer, and returns a `std::vector<Token>` containing token category, exact lexeme, source line and column. Tests cover keyword and identifier classification, numbers, quoted literals and recovery, operators, unknown characters, positions, and an actual file pipeline.

1.3 Explanation for a professor

This is a deliberately scoped lexical front-end, **not a complete C++ compiler frontend**. The preprocessor uses five states (`Normal`, `LineComment`, `BlockComment`, `StringLiteral`, `CharLiteral`) and

replaces supported comment bytes with spaces while preserving line feeds. The tokenizer is an ordered character scanner: it applies literal, identifier, numeric, punctuation, multi-character operator, single-character operator, and unknown-character rules, emitting typed tokens with 1-based locations. A modular `main()` delegates application control to `runCloneScope()`. Correctness for selected scenarios is demonstrated through independent module smoke tests, cross-module pipeline tests and manual file/folder execution.

1.4 Input, transformation, output

Stage	Input	Output	Responsibility
File reader	User path	Source text (`std::string`)	Open and read file contents
Preprocessor	Source text	Comment-masked text	Protect supported quoted text and preserve positions
Tokenizer	Comment-masked text	`std::vector<Token>`	Classify and locate recognized lexemes
Application	Token vector	Displayed count	Coordinate processing, not compare submissions

Important: A displayed count such as `Total tokens: 28` proves that a file was processed, not that *every* token has perfect C++ semantics. The pipeline test therefore checks selected types and lexemes in addition to the count.

1.5 Architecture map

```
User types a path
        |
        v
src/main.cpp: main()
        |
        v
src/application.cpp: runCloneScope()
        |
        +--> Check directory? --> Scan immediate .c/.cpp entries
        |                         or read one specified file
        v
std::ifstream + std::ostringstream
        |
        v
src/preprocessor.cpp: removeComments(sourceCode)
        |
        v
src/tokenizer.cpp: tokenize(cleanedCode)
        |
        v
std::vector<Token> [type, lexeme, line, column]
        |
        v
Print per-file token count
```

The architecture is intentionally **modular**. `main.cpp` is minimal, honoring the supervisor's expectation that it starts the program instead of owning the processing logic. Some file/folder workflow code remains duplicated inside `application.cpp`; that is a code-quality consideration for a later, controlled refactor, not evidence that tokenization failed.

2. Essential vocabulary

2.1 Source, character, lexeme, token

- **Source text:** The complete characters read from a `.c` or `.cpp` file. Example: `int x = 10;`.
- **Character:** One byte/character in the simplified scanner's input. The code reads it with `source[index]`.
- **Lexeme:** The exact substring of source text represented by a token. Examples: `"int"`, `"x"`, `"="`, `"10"`, `";"`.
- **Token:** A structured object consisting of a type, a lexeme, and source coordinates.
- **Token category:** A classification such as `Keyword`, `Identifier`, or `Operator`.
- **Lexical analysis:** The process of identifying token boundaries and categories from characters.
- **Preprocessing:** In this project milestone, specifically *basic comment masking*; not full C/C++ macro processing.
- **Source position:** Line number and column number where a token begins; positions are counted from **1**, unlike vector indexes.
- **Malformed literal:** A quote beginning that is not completed according to the supported ordinary-literal rules before a newline/end of input.

2.2 Why code comparison needs tokens

Consider two snippets:

```
int total = a + b;
```

```
int total=a+b;
```

The strings differ in spaces, yet their meaningful token sequences are the same:

```
int | total | = | a | + | b | ;
```

Tokenization provides a cleaner representation for later code-clone analysis. **It does not by itself decide whether the submissions are clones**, because the project must still define normalization, comparison, thresholds, tagging, reporting, and human review.

2.3 Taxonomy in this project

Category	Recognized example	Meaning
`Keyword`	`int`, `return`, `while`	Selected reserved words listed in this implementation
`Identifier`	`counter`, `_value`, `main`	Names starting with letter/underscore, followed by alphanumeric/underscore
`Number`	`123`, `3.14`, `0.5`	Supported basic decimal integers / fractional forms
`StringLiteral`	`"hello"`	Supported complete ordinary double-quoted text
`CharLiteral`	`'A'`, `'\\n'`	Supported complete ordinary single-quoted text

Category	Recognized example	Meaning
`Operator`	`+`, `==`, `::`, `->`, `.`	Supported one-/two-character operators
`Punctuation`	`(`, `)`, `{`, `}`, `;`	Supported structural separators
`Unknown`	`@`, unsupported `#`, unclosed `"abc`	Input not matched by current lexical rules

`Unknown` does not necessarily mean the original C/C++ program is illegal. For example `#` in `#include` is meaningful to the actual preprocessor but intentionally unsupported as a complete directive in this milestone.

3. C++17 concepts needed to read this implementation

3.1 Header vs implementation file

A *header* declares what an interface exposes; an implementation `.cpp` supplies executable definitions.

```
// tokenizer.hpp
std::vector<Token> tokenize(const std::string& source);
```

```
// tokenizer.cpp
std::vector<Token> tokenize(const std::string& source) {
    // ... scanner logic ...
}
```

The difference matters at link time. A translation unit may see a function declaration while the linker still needs the corresponding `.cpp` definition.

3.2 `#include` and `#pragma once`

```
#pragma once
#include <string>
#include <vector>
```

`#pragma once` asks the compiler to process this header once per translation unit and avoids repeated definitions. `#include <string>` provides `std::string`; `<vector>` provides `std::vector`; `<unordered_set>` provides the hash-set container; `<cctype>` supplies `std::isalpha`, `std::isdigit`, `std::isalnum`, `std::isspace`.

The project uses `#include "tokenizer.hpp"` for a local header and `#include <cctype>` for a standard library header. The quotes and angle brackets convey different search conventions.

3.3 `enum class` represents a controlled category

```
enum class TokenType {
    Keyword, Identifier, Number, StringLiteral,
    CharLiteral, Operator, Punctuation, Unknown
};
```

An `enum class` creates a scoped set of named values. `TokenType::Keyword` is more expressive and less error-prone than storing category `1` or a free-form string. The `::` here is C++ **scope resolution**, which the tokenizer itself also recognizes as an operator in source input.

Viva point: A strongly typed scoped enum prevents accidentally mixing token types with arbitrary integers.

3.4 `struct Token` combines related data

```
struct Token {
    TokenType type;
    std::string lexeme;
    std::size_t line;
    std::size_t column;
};
```

- `type` tells us the classification.
- `lexeme` preserves the original spelling of the token.
- `line` and `column` preserve the token's **starting** location.
- `std::size_t` is the standard unsigned size/index type used by strings and vectors.

One real token might conceptually be:

```
{ type: Keyword, lexeme: "int", line: 2, column: 1 }
```

3.5 `std::vector<Token>` stores an ordered sequence

```
std::vector<Token> tokens;
tokens.push_back(Token{TokenType::Operator, "+", 1, 7});
```

A vector grows as tokens are discovered. `push_back` appends one token, keeping source order. `Token{...}` is **aggregate initialization**: values are supplied in the field order of the `Token` struct.

Index vs coordinate: `tokens[0]` is the *first token* (index zero), while `tokens[0].column` may be `1` or another 1-based character position.

3.6 `const std::string& source`

```
std::vector<Token> tokenize(const std::string& source);
```

- `std::string`: a sequence of characters.
- `&`: a reference, so callers do not need to copy the entire string to pass it in.
- `const`: the function promises not to modify the input through this reference.
- Return type `std::vector<Token>`: caller receives a full token sequence.

Contrast `source.substr(start, length)`: unlike the input reference, `substr` **creates a new string** containing the requested characters.

3.7 `std::unordered_set` and membership checks

```
static const std::unordered_set<std::string> keywords = {
    "int", "float", "double", "char", "void",
    "if", "else", "for", "while", "return",
    "bool", "const", "class", "struct"
};
```

- `unordered_set` holds unique strings, optimized for hash-based membership queries.
- `static` gives the local object lifetime across calls rather than rebuilding it on each call.
- `const` means the supported set is not modified by the tokenizer.
- `.find(lexeme) != .end()` means the string was found.
- Average hash lookup is constant time; the C++ standard does not promise constant time for every pathological case.

3.8 Important control-flow syntax

Syntax	Meaning in this scanner
`while (index < source.size())`	Continue as long as source characters remain
`if (...) { ... }`	Select a lexical rule when its condition is true
`else if`	Apply alternative rule if earlier condition was false
`++index`	Move to next character
`index += 2`	Move past a two-character operator or an escape pair
`break`	Exit the *nearest loop*
`continue`	Begin the next iteration of the *nearest loop*
`return tokens`	Finish scanning and provide collected tokens
`&&`	Logical AND; conditions all need to be true
`||`	Logical OR; at least one condition must be true
`!`	Logical NOT, e.g. not whitespace

These details explain why placement of braces matters. Accidentally nesting a second string-scanning `while` inside an old one previously risked a loop that could stop making progress at a newline; the fix was to **replace** the original loop, not wrap it.

3.9 Character classification and safe conversion

```
unsigned char ch = static_cast<unsigned char>(current);
if (std::isalpha(ch) || current == '_') { ... }
```

Standard `<cctype>` functions require an input representable as `unsigned char` (or `EOF`). On implementations with signed `char`, directly passing a negative byte to `std::isalpha` can be undefined behavior. The cast makes the call safe on byte values. This **does not** provide full Unicode or UTF-8-aware C++ lexical analysis.

3.10 `std::string::npos`

```
if (punctuation.find(current) != std::string::npos) {
    // current is one of the supported punctuation characters
}
```

`find()` returns the position of a match or `std::string::npos` when no match exists. Comparing with `npos` is an idiomatic way to test membership in the small string of punctuation/operator characters.

4. Project files and responsibilities

File	What it owns	What it should *not* own
`src/main.cpp`	Starts application and returns exit code	Scanner implementation
`src/application.hpp`	Declares `runCloneScope()`	Large workflows
`src/application.cpp`	Path interaction, file/folder workflow, count display	Full tokenizer logic
`src/preprocessor.hpp`	Declares `removeComments()`	Token categories
`src/preprocessor.cpp`	Five-state comment masking	Clone scoring
`src/tokenizer.hpp`	`TokenType`, `Token`, `tokenize()` interface	File I/O

File	What it owns	What it should *not* own
`src/tokenizer.cpp`	Rule-based token recognition and source positions	Git or GUI behavior
`tests/preprocessor_smoke.cpp`	Preprocessor assertions	Production application entry point
`tests/tokenizer_smoke.cpp`	Lexical rule assertions	Production application entry point
`tests/pipeline_smoke.cpp`	File read + preprocessor + tokenizer assertions	Pairwise clone detection

4.1 Minimal `main()` is intentional

```cpp
#include "application.hpp"

int main()
{
    return runCloneScope();
}
```

Line by line:

1. Include the declaration of `runCloneScope`.
2. Define the process entry point `main`.
3. Call `runCloneScope()`; execution moves into the application module.
4. Return the integer exit status from that function to the operating system.

The earlier refactor moved the longer workflow into `application.cpp`. A linker error (`multiple definition of 'main'`, `undefined reference to 'runCloneScope()'`) occurred when copied code had not been saved. This is a valuable debugging lesson: source files on **disk**, not unsaved editor buffers, are compiled by `g++`.

4.2 The application interface

```cpp
#pragma once

// Start and coordinate the CloneScope application.
// Returns an exit status to the operating system.
int runCloneScope();
```

This makes it possible for `main.cpp` to call `runCloneScope()` without embedding file I/O details.

5. File reading and folder scanning

5.1 Getting the path

```cpp
std::string filePath;
std::cout << "Enter source file path: ";
std::getline(std::cin, filePath);
std::filesystem::path sourcePath(filePath);
```

`std::getline` reads the entire line, including spaces, until Enter. `std::filesystem::path` represents a filesystem path in a platform-oriented way. The project targets Windows PowerShell as the working terminal.

5.2 Selecting folder mode

```
if (fs::is_directory(sourcePath)) {
    for (const auto& entry : fs::directory_iterator(sourcePath)) {
        if (entry.is_regular_file() &&
            (entry.path().extension() == ".c" ||
             entry.path().extension() == ".cpp")) {
            // open and process entry
        }
    }
    return 0;
}
```

- `fs` is an alias for `std::filesystem` established in the application.
- `fs::is_directory` chooses folder behavior.
- `directory_iterator` visits **immediate children only**, not nested subfolders.
- `const auto& entry` avoids unnecessary entry copies and keeps entries read-only.
- `entry.is_regular_file()` filters out directories and other special entries.
- `extension()` is compared to lowercase `.c` / `.cpp`, so `.C`, `.CPP`, and `.h` are not included by these rules.
- The original file/folder workflow has separate processing branches; a future helper extraction could reduce duplication, but the implemented milestone works with this structure.

5.3 Opening and reading an entire file

```
std::ifstream inputFile(sourcePath);
if (!inputFile.is_open()) {
    std::cerr << "Error: Could not open source file.\n";
    return 1;
}
std::ostringstream buffer;
buffer << inputFile.rdbuf();
std::string sourceCode = buffer.str();
```

`std::ifstream` is an input file stream, `is_open()` checks whether the stream opened, `rdbuf()` exposes the stream's buffer, and `std::ostringstream` accumulates that input into a string. The file-reading approach is suitable for the small/student source inputs tested. Large files and filesystem exceptions have not been stress-tested in these sessions.

5.4 The processing hand-off

```
std::string cleanedCode = removeComments(sourceCode);
std::vector<Token> tokens = tokenize(cleanedCode);
std::cout << "\nTotal tokens: " << tokens.size() << '\n';
```

1. `removeComments` masks supported comments, preserving the surrounding code.
2. `tokenize` consumes the masked text and returns ordered token objects.
3. `.size()` reports total emitted tokens, including `Unknown` tokens.

The folder branch repeats these steps for each eligible source file. Source files are not concatenated; preserving file boundaries matters for later explainable comparisons.

6. Preprocessor state machine: concept first

6.1 Why a simple search-and-replace is incorrect

Consider:

```
std::string website = "https://example.com";
std::string notAComment = "/* still text */";
char slash = '/';
// This line really is a comment.
```

A naive removal rule that erases everything after `//` would destroy the URL string. Erasing everything between `/*` and `*/` could damage the string literal. The scanner therefore remembers **what kind of region** it is currently inside.

6.2 The five states

```
enum class ScanState {
    Normal,
    LineComment,
    BlockComment,
    StringLiteral,
    CharLiteral
};
```

State	Meaning	Typical way it ends
`Normal`	Ordinary source code	Transitions on comment/quote openers
`LineComment`	Between `//` and the next LF	Newline
`BlockComment`	Between `/*` and `*/`	Closing `*/`
`StringLiteral`	Ordinary double-quoted content	Closing `"` or recovery at newline
`CharLiteral`	Ordinary single-quoted content	Closing `'` or recovery at newline

This state machine is **finite-state lexical logic**, not a C++ parser: it doesn't build an abstract syntax tree or validate C++ statements.

6.3 State transition sketch

```
NORMAL -- sees // --> LINE_COMMENT -- LF --> NORMAL
   |                          \-- other chars --> spaces
   +-- sees /* --> BLOCK_COMMENT -- sees */ --> NORMAL
   |                                  \-- LF --> keep LF
   +-- sees "  --> STRING_LITERAL -- closing " --> NORMAL
   |                                  \-- physical LF --> NORMAL
   +-- sees '  --> CHAR_LITERAL   -- closing ' --> NORMAL
                                      \-- physical LF --> NORMAL
```

Escapes matter: A backslash followed by a supported next character within a quoted literal is copied together. This prevents escaped quotations such as `\"` from being treated as a closing delimiter.

6.4 Why mask comments instead of deleting bytes?

Suppose original code is:

```
int a; /* note */ int b;
```

`int b` starts at **column 19**. The scanner substitutes spaces for the comment bytes. If it deleted the comment completely, `int b` would shift left and source coordinates in future evidence might be misleading. It similarly preserves `\n` characters inside comments so later tokens keep their line numbers.

Position invariant for supported inputs: The output keeps the same length and LF positions as the input because every consumed non-LF byte is replaced by one output character and every consumed two-character delimiter is replaced by two spaces. This claim is about the supported basic scanner behavior, not a fully standards-compliant C++ preprocessing phase.

7. Preprocessor implementation: line-by-line reasoning

7.1 Function signature and variables

```
std::string removeComments(const std::string &source)
{
    ScanState state = ScanState::Normal;
    std::string result;
    for (std::size_t i = 0; i < source.size(); ++i)
    {
        char current = source[i];
        // ... state-dependent handling ...
    }
    return result;
}
```

Line/expression	What it does	Why it matters
`std::string removeComments(...)`	Returns processed text	Keeps preprocessing separate from file I/O
`const std::string &source`	Reads input by const reference	Avoids an unnecessary input copy
`ScanState state = ...Normal`	Initializes lexical context	At file start, not yet in a comment/quote
`std::string result`	Accumulates output	Holds the position-preserving masked source
`for (std::size_t i = 0; ...)`	Visits source positions	Needs lookahead for two-character delimiters
`char current = source[i]`	Reads current byte	Simple, explicit scanning
`return result`	Yields output for tokenizer	Makes pipeline composition possible

7.2 Detecting a `//` opener

```
if (current == '/' && i + 1 < source.size() &&
    source[i + 1] == '/')
{
    state = ScanState::LineComment;
    result += "  ";
```

```
    ++i;
}
```

Line by line: `current == '/'` checks the first slash. `i + 1 < source.size()` prevents reading beyond the string's end. `source[i + 1] == '/'` recognizes the second slash. The state changes to `LineComment`, two literal spaces replace the two delimiter characters, and `++i` skips the second slash because it has already been processed as part of the pair.

Critical note: Without the bounds check, `source[i + 1]` could be out of range when the last character is `/`.

7.3 Detecting `/*` and `*/`

```
else if (current == '/' && i + 1 < source.size() &&
         source[i + 1] == '*')
{
    state = ScanState::BlockComment;
    result += "  ";
    ++i;
}
```

The opening delimiter consumes two characters and produces two spaces. Later, while in `BlockComment`, a `*` followed by `/` creates two spaces and changes state back to `Normal`.

```
else if (state == ScanState::BlockComment)
{
    if (current == '*' && i + 1 < source.size() &&
        source[i + 1] == '/') {
        result += "  "; state = ScanState::Normal; ++i;
    }
    else if (current == '\n') { result += '\n'; }
    else { result += ' '; }
}
```

The newline branch is deliberately distinct: inserting a space instead of `\n` would destroy downstream line positions.

7.4 Entering quoted regions

```
else if (current == '"') {
    state = ScanState::StringLiteral;
    result += current;
}
else if (current == '\'') {
    state = ScanState::CharLiteral;
    result += current;
}
```

The opening quotation marks are **kept**, because literals are meaningful program data, not comments. Inside these states, a `//` sequence must be copied as literal content rather than interpreted as the start of a comment.

7.5 Ending line comments

```
else if (state == ScanState::LineComment)
{
    if (current == '\n') {
```

```
        result += '\n';
        state = ScanState::Normal;
    } else {
        result += ' ';
    }
}
```

The line comment ends at LF. Preserving the LF resets the tokenizer's later `line/column` tracking correctly. On Windows, input may contain CRLF (`\r\n`); here the CR is masked inside comments and LF retained. Actual C++ standard preprocessing behavior is broader than this simplified byte scanner.

7.6 Protecting escape pairs inside quotes

```
else if (current == '\\' &&
         i + 1 < source.size() &&
         source[i + 1] != '\n')
{
    result += current;
    result += source[i + 1];
    ++i;
}
```

The first condition asks whether the current byte is backslash. The bounds check makes lookahead safe. The final check avoids crossing a physical LF as part of the simplified escape pair. Both backslash and its following character are copied to the result; `++i` skips the second byte. This is the *actual behavior used in the project*, not a complete implementation of preprocessing line splices.

7.7 Malformed ordinary-literal recovery

```
if (current == '\n')
{
    result += '\n';
    state = ScanState::Normal;
}
```

This logic is now present in **both** `StringLiteral` and `CharLiteral` states. Its purpose is recovery: a missing closing quote must not cause the next line's `// comment` to be preserved as quoted text. The pipeline tests deliberately reproduced and then verified fixes for this bug.

7.8 What this preprocessor does not do

It doesn't expand macros, handle `#define`, parse directives, process raw strings (`R"(...)"`) correctly in general, or fully implement all physical-line splicing/CRLF edge cases. It does not validate the entire C++ language. The documentation should state these limits rather than silently claim more coverage.

8. Tokenizer architecture: the main scanning loop

8.1 Interface and initial state

```
std::vector<Token> tokenize(const std::string& source)
{
    std::vector<Token> tokens;
```

```
    std::size_t index = 0;
    std::size_t line = 1;
    std::size_t column = 1;

    while (index < source.size()) {
        char current = source[index];
        // Ordered recognition rules follow.
    }
    return tokens;
}
```

How to explain every variable: `tokens` is the growing output sequence; `index` selects the next unread source character; `line` and `column` locate that character in a human-readable 1-based coordinate system; `current` is the character at `index` in the current iteration.

The **progress rule** is vital: every branch that uses `continue` must have advanced `index` or otherwise made progress before the next outer-loop iteration. If a scanner stops at a newline, the inner scanner leaves the newline for the outer loop, which eventually increments `index`.

8.2 Rule order and why it matters

The reviewed implementation checks approximately this order:

1. Opening `"`: parse a double-quoted ordinary literal.
2. Letter or _: scan identifier-shaped word, then check keyword set.
3. Digit: scan decimal-number form.
4. Opening `'`: parse a single-quoted ordinary character literal.
5. Structural punctuation: recognize `(){}[],;`.
6. Supported **two-character** operator.
7. Supported **one-character** operator.
8. Unsupported non-whitespace byte: emit `Unknown`.
9. Advance source position; LF increments line and resets column.

The rule order ensures `3.14` is consumed by the numeric rule before `.` could be classified as an operator. Two-character operators such as `->` must be checked before single-character `-` and `>`.

8.3 Keyword recognition table

The reviewed keyword set contains exactly these 14 selected words:

```
int, float, double, char, void,
if, else, for, while, return,
bool, const, class, struct
```

Other real C++ keywords may currently be tokenized as `Identifier`. This is a **documented scope limit**, not a reason to claim the source program is wrong.

9. Scanning identifiers and keywords

```
unsigned char ch = static_cast<unsigned char>(current);
if (std::isalpha(ch) || current == '_') {
    std::size_t start = index;
    std::size_t startColumn = column;

    while (index < source.size()) {
        unsigned char next =
            static_cast<unsigned char>(source[index]);
```

```
        if (!std::isalnum(next) && source[index] != '_') {
            break;
        }
        ++index;
        ++column;
    }

    std::string lexeme = source.substr(start, index - start);
    TokenType type = TokenType::Identifier;
    if (keywords.find(lexeme) != keywords.end()) {
        type = TokenType::Keyword;
    }
    tokens.push_back(Token{type, lexeme, line, startColumn});
    continue;
}
```

9.1 Statement-by-statement commentary

Statement	What?	Why?
`std::isalpha(ch) || current == '_'`	Accept an identifier beginning	`_value` is a valid identifier-shaped lexeme
`start = index`	Save first character index	Needed to extract complete substring
`startColumn = column`	Save starting column	`column` changes during scanning
`while (index < source.size())`	Consume subsequent characters	Names can be longer than one byte
`std::isalnum(next)`	Check letter or digit continuation	Supports names like `total_2`
`! ... && source[index] != '_'`	Stop on nonname character	Prevents swallowing punctuation
`break`	Exit name-consuming loop	Leaves next separator for outer loop
`source.substr(start,index-start)`	Extract the lexeme	Keeps original spelling
`TokenType::Identifier`	Default category	Most names are not selected keywords
`keywords.find(...)`	Check reserved-word set	Classify `int` differently from `main`
`push_back(...)`	Emit token with saved location	Builds the ordered result
`continue`	Restart top-level scanner	Avoids falling into later rules for same text

9.2 Worked trace: `int total_2;`

```
Columns: 123456789012
Input:   int total_2;
Tokens:  [int] [total_2] [;]
```

Lexeme	Type	Start column	Explanation
`int`	Keyword	1	Name-shaped lexeme found in keyword set
`total_2`	Identifier	5	Not in keyword set; _/digit allowed after start
`;`	Punctuation	12	Handled by punctuation rule

The tokenizer does not emit a token for the separating space. It advances `column` over it.

10. Scanning decimal numbers

```
if (std::isdigit(ch)) {
    std::size_t start = index;
    std::size_t startColumn = column;
    while (index < source.size()) {
        unsigned char next =
            static_cast<unsigned char>(source[index]);
        if (!std::isdigit(next)) break;
        ++index;
        ++column;
    }
    if (index + 1 < source.size() &&
        source[index] == '.' &&
        std::isdigit(static_cast<unsigned char>(source[index + 1]))) {
        ++index; ++column;  // decimal point
        while (index < source.size()) {
            unsigned char next =
                static_cast<unsigned char>(source[index]);
            if (!std::isdigit(next)) break;
            ++index; ++column;
        }
    }
    tokens.push_back(Token{
        TokenType::Number, source.substr(start, index-start),
        line, startColumn});
    continue;
}
```

10.1 Why there are two digit loops

The first loop consumes the integral part (`3` in `3.14`). A conditional then checks a dot **followed by at least one digit**. If so, the second loop consumes the fractional digits (`14`). The test for `index + 1 < source.size()` prevents lookahead past end-of-input.

10.2 What works vs what doesn’t

Input	Current simplified behavior	Caution
`123`	One `Number` token	Supported
`3.14`	One `Number` token	Supported
`0.5`	One `Number` token	Supported
`.5`	`.` operator, `5` number	Not normalized as one number
`1.`	`1` number, `.` operator	Not normalized as one number
`1e3`	`1` number, `e3` identifier	Scientific form unsupported
`0xFF`	May split into `0` + `xFF`	Hex form unsupported
`42u`	Number + identifier	Suffix unsupported

Viva caution: Don't say "recognizes all C++ numeric literals." Say "recognizes basic decimal integral and fractional forms".

11. Quoted literals and escape sequences

11.1 String-literal rule

```
if (current == '"') {
    std::size_t start = index;
    std::size_t startColumn = column;
    ++index; ++column;  // opening quote

    while (index < source.size() &&
           source[index] != '"' &&
           source[index] != '\n') {
        if (source[index] == '\\' &&
            index + 1 < source.size() &&
            source[index + 1] != '\n') {
            index += 2;
            column += 2;
        } else {
            ++index;
            ++column;
        }
    }

    TokenType type = TokenType::Unknown;
    if (index < source.size() && source[index] == '"') {
        ++index; ++column;
        type = TokenType::StringLiteral;
    }
    tokens.push_back(Token{
        type, source.substr(start, index-start),
        line, startColumn});
    continue;
}
```

Why save `start` and `startColumn`? We need the entire quoted text as one lexeme while reporting the position of its opening quotation mark, not its closing quotation mark.

Why default to `Unknown`? A closing quotation mark has not yet been proven to exist. Only if the post-loop condition finds the actual correct closing delimiter do we upgrade the token to `StringLiteral`.

Why stop at `\n`? According to this simplified ordinary-string rule, an unterminated literal must not consume later source lines. The newline is left unconsumed for the outer scanner.

11.2 Character-literal rule

The `CharLiteral` branch follows nearly the same algorithm, but uses single quotations (`'`). It recognizes examples such as `'A'`, `'\\n'`, and escaped apostrophes within the supported basic scanning model. The code checks for a closing `'` explicitly before classifying a complete character token.

11.3 Three different backslash cases

Text in a C++ *test string expression*	Actual characters received by tokenizer	Meaning
`"\\n"`	backslash then letter `n`	Two visible source characters
`"\n"`	actual line-feed	One physical newline
`"\\\""` (inside a larger escaped test source)	backslash + quote	Escaped quote in source being tokenized

The key distinction: a newline escape **inside a string literal in the test program** is not the same as an actual newline character within the source string passed to the tokenizer. The test suite includes both valid escaped literals and physically unterminated literals.

11.4 Escaped quote example

Input to tokenizer (shown as source text):

```
"a\"+b"
```

The scanner consumes the sequence `\"` as a pair instead of mistaking its quote for the string terminator, then continues until the final closing `"`. The entire quoted text remains a **single** `StringLiteral` token.

11.5 Malformed ordinary literals

Input:

```
"unfinished
int x
```

Expected simplified tokens:

Index	Lexeme	Type	Line:column
0	`"unfinished`	Unknown	1:1
1	`int`	Keyword	2:1
2	`x`	Identifier	2:5

A nearly identical test exists for an unterminated character literal (`'A`). Both conditions were verified through smoke tests; preprocessor recovery was separately tested through the file pipeline.

12. Punctuation, operators, and the longest supported match

12.1 Structural punctuation

```
static const std::string punctuation = "(){}[],;";
```

For each recognized punctuation character, the scanner emits one `Punctuation` token, advances index and column by 1, and continues. These symbols preserve structure like blocks, function calls, argument lists and statement endings.

12.2 One-character operators

The reviewed code uses:

```
static const std::string singleCharOperators = "+-*/%&=!<>|^~.";
```

A standalone `.` reaches this rule **only if** it has not already been consumed by numeric scanning. The dot is used for direct member access, e.g. `obj.field`.

12.3 Two-character operators

The reviewed set includes:

```
static const std::unordered_set<std::string> twoCharOperators = {
    "==", "++", "--",
    "+=", "-=", "*=", "/=",
    "%=", "&=", "|=", "^=",
    "!=", "<=", ">=", "&&", "||",
    "::", "->"
};
```

`::` preserves scope resolution (`std::cout`); `->` preserves pointer member access (`node->value`). The scanner constructs a two-character `pair`, searches the set, and consumes both bytes when a match exists.

12.4 Longest *supported* match

```
if (index + 1 < source.size()) {
    std::string pair = source.substr(index, 2);
    if (twoCharOperators.find(pair) != twoCharOperators.end()) {
        tokens.push_back(Token{TokenType::Operator,
                               pair, line, column});
        index += 2;
        column += 2;
        continue;
    }
}
```

Walkthrough for `node->value`:

1. Identifier rule consumes `node` (columns 1-4).
2. At column 5, `pair` becomes `"->"`.
3. `pair` exists in `twoCharOperators`; emit one `Operator` token at column 5.
4. Move `index` and `column` by 2, landing on `v` at column 7.
5. Identifier rule consumes `value`.

The code checks supported pairs before single-character operators. However, it has **no complete three-/four-character operator recognizer**. The phrase “longest-match” in this manual means longest *supported match in the implemented rules*, not complete C++ maximal-munch tokenization.

12.5 Worked operator trace

Input	Token sequence	Why
`std::cout`	`std`, `::`, `cout`	Scope resolution is a two-character operator
`node->value`	`node`, `->`, `value`	Pointer member access is one operator
`obj.field`	`obj`, `.`, `field`	Dot is a one-character operator
`a==b`	`a`, `==`, `b`	Pair checked before individual `=`
`x+=2`	`x`, `+=`, `2`	Compound assignment preserved
`3.14`	`3.14`	Number rule runs before dot rule

13. Unknown tokens, whitespace, and positions

13.1 Why unknowns must not vanish

The original fallback silently skipped unsupported bytes. If one file had `int x = 5;` and another had `int x @ 5;`, dropping `@` could obscure a difference. Instead, the project now emits an `Unknown` token for each unsupported non-whitespace character.

```
if (!std::isspace(static_cast<unsigned char>(current))) {
    tokens.push_back(Token{
        TokenType::Unknown,
        source.substr(index, 1),
        line, column
    });
}
```

`!std::isspace(...)` means “if this character is not whitespace.” The cast is needed for safe `<cctype>` usage. `substr(index,1)` extracts exactly the current character. Because the branch does not use `continue`, execution next reaches the shared position-update code.

13.2 Newline vs other whitespace

```
if (current == '\n') {
    ++line;
    column = 1;
} else {
    ++column;
}
++index;
```

- On LF (`\n`), the next character is at the start of the next line: increment line, reset column to 1.
- Otherwise, one character has been consumed on the current line: increment column.

- Always increment `index` after this fallback.

Current caveat: This scanner's location semantics are byte-based and centered on LF. Full CRLF/newline normalization, tabs as display widths, and Unicode-aware display columns are not guaranteed. For typical ASCII/LF cases tested, the recorded positions are consistent.

13.3 Worked example including unknown

Input:

```
int x @ 5;
```

Vector index	Lexeme	Type	Column
0	`int`	Keyword	1
1	`x`	Identifier	5
2	`@`	Unknown	7
3	`5`	Number	9
4	`;`	Punctuation	10

This is precisely what the `unknownTokens` regression assertions test.

14. Full worked source trace

Consider the mixed snippet used in `tokenizer_smoke.cpp`:

```
int main() {
    double rate = 3.14;
    node->value = rate;
    return 0;
}
```

The test deliberately treats `node` as an identifier for lexical testing; this snippet is not meant to compile alone as a C++ program.

14.1 Expected tokens by line

Line	Tokens (source order)	Count
1	`int`, `main`, `(`, `)`, `{`	5
2	`double`, `rate`, `=`, `3.14`, `;`	5
3	`node`, `->`, `value`, `=`, `rate`, `;`	6
4	`return`, `0`, `;`	3
5	`}`	1
Total		**20**

14.2 Selected index-to-coordinate checks

Index	Lexeme	Type	Location
0	`int`	Keyword	1:1
8	`3.14`	Number	2:19
11	`->`	Operator	3:9
16	`return`	Keyword	4:5

Index	Lexeme	Type	Location
19	`}`	Punctuation	5:1

How can token index 8 be at column 19? Because token index is counted across the vector from **0**, while column is the character's position within **line 2**. The numeric rule tracks `startColumn` before consuming all digits.

14.3 Pencil-and-paper tracing method

For any snippet, draw a four-column table: `index`, `current`, `line`, `column`. At each outer-loop step:

1. Record `current = source[index]`.
2. Determine which recognition branch matches first.
3. Save start coordinates if a multi-character lexeme begins.
4. Trace index increments (`+1` or `+2`) inside the branch.
5. Write the emitted token.
6. Ensure the next unread source character is still available for the outer loop.

Try the exercise on `int x=3.14;` and explain why `.` is *not* separately emitted.

15. Automated tests: what was actually verified

15.1 Independent preprocessor smoke test

`tests/preprocessor_smoke.cpp` was run with `src/preprocessor.cpp` and produced PASS lines for:

1. Single-line comment removal.
2. Block-comment removal.
3. String-literal preservation.
4. Escaped quote preservation.
5. Character-literal preservation.

These tests confirm supported cases; they do not certify complete C++ preprocessing.

15.2 Independent tokenizer smoke test

`tests/tokenizer_smoke.cpp` was tested with `src/tokenizer.cpp`. Reviewed assertions cover:

- Keyword/identifier classification and source coordinates.
- Decimal integers and supported fractional numbers.
- Structural punctuation.
- Selected single- and two-character operators, including compound assignment and logical/comparison operators.
- `::`, `->`, and `..`
- Complete ordinary string/character literals and supported backslash escape pairs.
- Unterminated ordinary strings/chars at end-of-file or LF.
- Unsupported non-whitespace characters as `Unknown`.
- Mixed five-line snippet with 20 emitted tokens and selected locations.

The success message is:

```
All tokenizer smoke tests passed.
```

15.3 Automated file pipeline smoke test

The added `tests/pipeline_smoke.cpp` reads `samples/hello.c`, calls `removeComments`, then `tokenize`, and checks:

- The file opens from the repository root.
- The current fixture yields **28** tokens.
- Selected tokens have expected type and exact lexeme (`int` keyword, a `printf` string).
- The first `#` in `#include` is emitted as `Unknown` under this limited tokenizer.
- Comment masking preserves selected positions (`int` at line 1, col 19 after a block comment; later `int` at line 3, col 1 after a line comment).
- Malformed ordinary string/character literals recover at LF and allow the next-line comment to be masked.

The success message is:

```
File pipeline smoke test passed.
```

15.4 Why assert *both* count and content?

A test `assert(tokens.size()==28)` could pass even if the 28 tokens were incorrectly classified. Additional checks such as `assert(tokens[19].type==TokenType::StringLiteral)` provide stronger evidence. A robust future test suite could check every token and its position, but the reviewed milestone uses selected high-value assertions.

15.5 Red-green regression example

Red: A new test for `"unfinished\n// comment\nint x;` initially failed at `recoveryTokens.size()==4` because the preprocessor remained in `StringLiteral` state beyond LF.

Fix: Modify the preprocessor so ordinary `StringLiteral` state preserves LF and returns to `Normal`. Ensure a backslash pair does not skip physical LF.

Green: Recompile and run the **unchanged** test. The file-pipeline test passed. The same pattern was then repeated for `CharLiteral` state.

This demonstrates test-first debugging and the value of keeping tests when fixing bugs.

15.6 `assert` caveat

`assert(condition)` terminates the program when a condition is false in ordinary debug/test builds. If `NDEBUG` is defined, C/C++ may compile these assertions out. The shown project build commands do **not** define `NDEBUG`. For a more mature test framework, explicit failure reporting can avoid relying solely on assertions, but it is outside the current SPL-1 smoke-test design.

16. Exact Windows PowerShell build and run reference

Run from the repository root (`SPL_01_BSSE1708`), not from an arbitrary working directory.

16.1 Preprocessor test

```
g++ -std=c++17 -Wall -Wextra tests/preprocessor_smoke.cpp src/preprocessor.cpp -o preprocessor_smoke.exe
.\preprocessor_smoke.exe
```

16.2 Tokenizer test

```
g++ -std=c++17 -Wall -Wextra tests/tokenizer_smoke.cpp src/tokenizer.cpp -o tokenizer_smoke.exe
.\tokenizer_smoke.exe
```

16.3 File pipeline test

```
g++ -std=c++17 -Wall -Wextra tests/pipeline_smoke.cpp src/preprocessor.cpp src/tokenizer.cpp -o pipeline_sm
.\pipeline_smoke.exe
```

16.4 Complete application

```
g++ -std=c++17 -Wall -Wextra src/main.cpp src/application.cpp src/preprocessor.cpp src/tokenizer.cpp -o clo
.\clonescope.exe
```

At `Enter source file path:`, use `samples/hello.c` for one file or `samples` for the immediate folder. The reviewed manual runs displayed **28 tokens** for the sample C file and **45 tokens** for the sample `preprocess_example.cpp` file in folder mode. These counts are **observed fixture-specific results**, not universal numbers.

16.5 Why each `.cpp` belongs in a command

Program being built	Files that define required functions
Preprocessor smoke test	Its own test `main` + `removeComments`
Tokenizer smoke test	Its own test `main` + `tokenize`
Pipeline smoke test	Its own test `main` + `removeComments` + `tokenize`
Production CloneScope	Production `main` + `runCloneScope` + both processing modules

Never add both a test `.cpp` containing `main()` **and** `src/main.cpp` to one executable command. That produces a multiple-definition error for `main`.

16.6 Compiler switches explained

- `g++`: GNU C++ compiler driver, also invokes linker.
- `-std=c++17`: Compile with C++17 language rules (needed for `std::filesystem`).
- `-Wall`: Enable a useful collection of warnings.
- `-Wextra`: Enable additional warnings.
- `-o clonescope.exe`: Choose output executable path.

Warnings are diagnostic feedback; zero warnings on a build does not establish mathematical correctness.

17. Git workflow and verified milestones

17.1 Why work on a feature branch?

Development took place on `feat/tokenization`, allowing the `main` branch to remain separate until tests and documentation are reviewed. The branch was deliberately not published/merged while implementation was incomplete.

17.2 Milestone commits observed during development

Short hash	Commit message	Scope
`d3fe422`	`feat: add quoted literal handling and preserve unknown tokens`	Ordinary quoted tokens, malformed handling, unknown preservation
`b44bc89`	`feat: integrate tokenizer with file and folder processing`	Operator additions, minimal `main`, application integration, mixed test
`3dc914c`	`test: add pipeline regression tests and fix literal recovery`	New pipeline tests, preprocessor malformed-literal recovery

These short IDs were reported by local Git logs in the reviewed screenshots. A complete commit history may contain additional earlier incremental operator/keyword commits.

17.3 Useful Git verification commands

```
git status --short
git branch --show-current
git diff --check
git diff --cached --check
git diff --cached --stat
git log -1 --oneline
```

- `git status --short`: Shows modified (`M`) and untracked (`??`) files.
- `git branch --show-current`: Confirms active branch before commit.
- `git diff --check`: Checks whitespace in unstaged tracked changes.
- `git diff --cached --check`: Checks staged content, including newly added files.
- `git diff --cached --stat`: Reviews scope of staged changes.
- `git log -1 --oneline`: Verifies latest commit and hash.

Use VS Code Source Control to stage/commit the intended files; compile/run in PowerShell as established in the development workflow. `.exe` files are build artifacts, not source-code deliverables; do not commit them.

17.4 What a safe PR review checks

Before merging into `main`: (1) clean working tree; (2) correct feature branch; (3) independent module tests passing; (4) pipeline test passing; (5) complete app can process expected samples; (6) README and learning manual accurately describe limitations; (7) staged/source diff reviewed; (8) correct upstream branch and PR target. A PR merge marks **tokenization milestone integration**, not completion of clone detection.

18. Complexity analysis and design tradeoffs

18.1 Time complexity

Let `n` be the number of input source characters.

Comment masking: The `for` loop visits characters once, occasionally consuming a delimiter/escape pair with `++i`. Time is **O(n)**. Constructing the output string uses **O(n)** extra space.

Tokenization: The scanner generally consumes each input character once. For fixed-size operator and keyword sets and expected constant-time hash membership, basic tokenization is **O(n)**

on typical inputs. Creating lexeme substrings copies characters, but the total emitted lexeme lengths are bounded by source length for the supported disjoint token scanning strategy. Strict pathological `unordered_set` performance can be worse than its average-case promise.

File reading: Reading the file into a string is **O(n)** time and **O(n)** memory. The application currently keeps source, cleaned source, and token outputs in memory; asymptotic extra memory is **O(n)**.

Folder mode: For files with lengths `n_1 ... n_k`, processing cost is approximately **O(sum n_i)** plus filesystem iteration and I/O overhead, under the same assumptions.

18.2 Why not regular expressions for everything?

A small ordered scanner with state supports explicit context (comments versus strings), deliberate recovery and positions. It also makes every transition easy to explain in an SPL viva. Regular expressions alone are not a drop-in replacement for context-sensitive comment masking with quotes and escapes.

18.3 Why separate preprocessing from tokenization?

- Easier to test a single responsibility independently.
- Comments do not create misleading token content when handled by the preprocessing stage.
- Separate outputs help locate bugs: is text already wrong after masking, or did the tokenizer misclassify it?
- Source locations can remain explainable when positions are preserved.

The separation also introduces a **consistency requirement**: both modules must agree on ordinary quote/newline recovery. The completed regression tests exposed and fixed a mismatch.

18.4 Tradeoffs of fixed lexical subsets

A fixed keyword/operator set gives transparent behavior and a manageable C++17 SPL-1 implementation. It does *not* automatically support every construct found in student projects. More language coverage would require carefully designed rules and tests; adding them merely to inflate project scope is not necessary for a truthful initial clone-analysis prototype.

19. Explicit limitations and risks to disclose

Limitation	Practical example	Reporting consequence
Not a full preprocessor	`#define`, macro expansion	Source may contain `Unknown` or unexpanded constructs
Incomplete numeric lexer	`.5`, `1e3`, `0x1F`	May split into multiple tokens
Incomplete operator set	`<<`, `>>`, `<<=`, `...`	May split or be partly `Unknown`
Raw string unsupported	`R"(text // text)"`	Basic comment masking may misinterpret content
Full line splicing unsupported	backslash followed by physical LF	Byte-level behavior differs from C++ translation phases
LF-centered position logic	mixed line endings, tabs	Display columns/line mapping need care
Simplified character validation	`''`, `'ab'`	Completeness check does not equal full C++ validity check
Nonrecursive folder scan	nested student submission directories	Nested files not visited

Limitation	Practical example	Reporting consequence
Lowercase `.c`, `.cpp` filter	`FILE.CPP`, headers	Excluded by current extension condition
Basic path/error handling	inaccessible folder, filesystem exceptions	No claim of full robust directory error recovery
Counts not evidence of clones	both files yield 28 tokens	Matching count proves no structural match

The ethical point: Token matches will eventually be *candidate evidence* for instructor review; a tokenizer output or future similarity score is not proof of misconduct. This aligns with the approved explainable, human-review-focused direction of CloneScope.

20. Supervisor viva: conceptual questions and model answers

Q1. What is tokenization? It is scanning source text to form meaningful lexical units (tokens), each labeled by category. CloneScope also saves source coordinates.

Q2. What is a lexeme? The exact source substring making up one token, such as `"return"` or `"=="`.

Q3. Why use `enum class TokenType`? Scoped named categories make token classification clearer and safer than arbitrary integer codes.

Q4. Why does `Token` include line and column? Later explainable reports must point a reviewer to the original code location.

Q5. Why are positions 1-based while vector indexes are 0-based? Humans refer to source line 1/column 1, but C++ indexed containers begin at index 0.

Q6. Why keep the lexeme text? It preserves the original spelling needed for inspection and later normalization choices.

Q7. Why check for identifiers before checking keywords? The scanner first finds a complete word-shaped lexeme, then tests membership in the selected keyword set.

Q8. Why use `unordered_set` for keywords? It gives readable membership logic and average constant-time lookup, with worst-case caveats.

Q9. What does `static const` accomplish here? A fixed local collection can be reused across calls without being reconstructed each time.

Q10. Why use `std::size_t` for indexes? It is the standard type for sizes and indexing in C++ containers and strings.

Q11. Why cast to `unsigned char` before `std::isdigit`? To satisfy `<cctype>` input requirements and avoid undefined behavior for negative signed-char values.

Q12. Why use `const std::string&`? It avoids copying large input strings and prevents mutation through that function parameter.

Q13. What does `source.substr(start,index-start)` do? It copies the complete recognized lexeme from its saved starting position to the current scanner index.

Q14. Why store `startColumn`? `column` advances while consuming a token; the token should report where it began.

Q15. What does `continue` do in the scanner? It restarts the outer scanning loop after a token was emitted, avoiding accidental processing by later rules.

Q16. What invariant prevents an infinite loop? The scanner must advance `index` or otherwise consume input on every outer-loop iteration.

Q17. Why is `::` one token? C++ scope resolution has one semantic operator spelling, so the pair is recognized as a unit.

Q18. Why is `->` one token? It represents pointer member access, not two independent operations in this context.

Q19. Why recognize two-character operators first? To prevent supported pairs such as `==` or `->` from being split into individual characters.

Q20. Why does `3.14` remain one token after adding dot operator? The numeric rule starts with a digit and consumes a supported fractional part before the standalone dot rule runs.

Q21. What does `Unknown` mean? Text that the current supported lexer rules did not classify; it is not necessarily syntactically illegal C++.

Q22. Why not drop unknown characters? Dropping source content can distort comparisons and hide differences between submissions.

Q23. Why skip whitespace? The tokenizer's current representation focuses on non-whitespace lexical units; location counters still advance over whitespace.

Q24. Why protect `//` inside strings? It is literal content, not a source comment opener in that context.

Q25. What is the preprocessor's state machine? It tracks whether the scanner is in ordinary code, line comment, block comment, string literal, or character literal.

Q26. Why replace comments with spaces? It retains supported source-coordinate mapping instead of shifting the remaining code.

Q27. Why preserve newlines within removed comments? Otherwise later token line numbers would be incorrect.

Q28. Why does an unterminated quote recover at LF? Under the project's simplified ordinary-literal rules, later lines should not be consumed as malformed quoted content.

Q29. Why separate preprocessor and tokenizer tests? A failing test can identify whether the defect is in comment handling or token scanning.

Q30. Why add a pipeline test? It verifies that reading, comment masking and tokenization work together on an actual source file.

21. Supervisor viva: implementation, testing, architecture and ethics

Q31. Why is `main()` so short? Its responsibility is only to start the application; `runCloneScope` coordinates the rest.

Q32. What is the difference between declaration and definition? A declaration states a function interface, while a definition provides its implementation.

Q33. Why did a linker say `multiple definition of main`? Both source files on disk appeared to define `main` during a partial refactor; after saving the renamed `runCloneScope` implementation, the link succeeded.

Q34. Why compile several `.cpp` files? The linker needs implementations of every called function across modules.

Q35. Why not compile `tests/pipeline_smoke.cpp` with `src/main.cpp`? Each defines its own `main`, so the executable would have duplicate entry-point definitions.

Q36. What does `-std=c++17` select? The C++17 language version, used here with `std::filesystem`.

Q37. Are warnings and runtime test passes sufficient for full correctness? No. They establish only the behavior exercised by the tests and sample runs.

Q38. Why use `assert` in smoke tests? It is a lightweight way to stop when a required condition is false during a debug/test build.

Q39. What is regression testing? Rerunning older tests after modifications to check that established behavior remains intact.

Q40. Explain red-green debugging. Write a test that demonstrates an existing failure (red), apply a fix, then rerun unchanged tests successfully (green).

Q41. Why create focused Git commits? They document coherent milestones and aid review, rollback and debugging.

Q42. Why keep `.exe` out of Git? Binaries are generated artifacts; source and tests are the reproducible development inputs.

Q43. What is different about single-file vs folder mode? Single-file mode handles one path; folder mode visits each immediate eligible `.c`/`.cpp` child independently.

Q44. Is folder scanning recursive? No, `directory_iterator` visits only immediate children in the current implementation.

Q45. Does this tokenizer implement complete C++ lexical analysis? No. It supports a documented subset; macros, raw strings, advanced numbers and more are incomplete.

Q46. Does the output token count measure similarity? No. Two files can have the same token count while containing completely different code.

Q47. Does this phase detect Type-1 and Type-2 clones? Not yet. It provides the tokens that later matching and normalization stages can use.

Q48. Is tokenization sufficient to accuse a student of plagiarism? No. CloneScope is designed to provide explainable candidate evidence requiring instructor review.

Q49. What is the next technical stage? Build function/block representations and later comparison/tagging logic within the approved project scope.

Q50. Summarize your architecture in ten seconds. "A minimal `main` calls the application layer, which reads C/C++ files, masks supported comments without shifting positions, and tokenizes the result into labeled lexemes with source locations."

22. Practice exercises and fully explained solutions

Exercise 1: Identify the token categories

Input: `int total_2 = 45;`

Solution: `int` -> Keyword; `total_2` -> Identifier; `=` -> Operator; `45` -> Number; `;` -> Punctuation. Five tokens. The spaces are not emitted but move the location counter.

Exercise 2: Why is `node->value` three tokens?

Solution: The name rule consumes `node`. At the next unread character the two-byte operator lookup matches `->`, so the scanner advances two columns and emits one operator. Finally `value` is an identifier. If `->` were not in the supported pair set, `-` and `>` could instead be emitted separately, losing the intended operator unit.

Exercise 3: Trace a physical newline

Input shown on two lines:

```
int x;
return x;
```

Solution: After `;` on line 1, the outer fallback sees LF, increments `line` from 1 to 2, resets `column` to 1, and moves `index` forward. The `return` keyword therefore starts at 2:1.

Exercise 4: Comment masking and columns

Input: `int a; /* note */ int b;`

Solution: The literal text `/* note */` contains ten characters, each replaced with a space by the supported preprocessor. The subsequent `int` starts at column 19 in both original and masked source. The integration regression test asserts that coordinate.

Exercise 5: Can the tokenizer recognize `.5` as one number?

Solution: No. The numeric scanner only starts if the current character is a digit; `.` is checked later as a standalone operator. The simplified output is dot operator followed by number `5`. It is a documented lexical limitation.

Exercise 6: Why would a missing quote change preprocessing?

Input:

```
"unfinished
// comment
int x;
```

Solution: Without newline recovery the preprocessor would remain in `StringLiteral` state when it sees `//`, so the second line could be copied rather than masked. The fix resets state to `Normal` at a physical LF. The pipeline regression test checks that only the malformed first-line token plus `int`, `x`, and `;` remain.

Exercise 7: Find the number of tokens

Input: `std::cout << x;`

Solution: Under the supported rules, `std`, `::`, `cout`, `<`, `<`, `x`, `;` are seven tokens. Because `<<` is not in the reviewed two-character operator set, it is split into two supported `<` operator tokens. This illustrates why it is important to document exactly which operator spellings are supported.

Exercise 8: Explain a linker error

Problem: `multiple definition of 'main'` and `undefined reference to 'runCloneScope()'`.

Solution: Review which `.cpp` files define `main` and ensure the application implementation defines `runCloneScope` with the signature declared in `application.hpp`. Save all editor buffers before building, then include exactly one production `main.cpp` when compiling the application. Independent tests have their own `main` and must be built separately.

23. Final pre-merge checklist

This checklist reflects how to *verify* the module, not a statement that a particular future GitHub PR is already merged.

- ☐ `src/main.cpp` only calls `runCloneScope()` and returns its status.
- ☐ Preprocessor, tokenizer and pipeline smoke tests compile and pass.
- ☐ Complete application compiles from all four production `.cpp` files.
- ☐ Single sample file can be processed; folder mode prints a count per eligible file.
- ☐ README lists actual supported features and known limitations, without claiming clone matching is finished.
- ☐ Unknown token behavior and malformed ordinary quote recovery are documented.
- ☐ All intended files are staged; no executable or unrelated change is staged.
- ☐ `git diff --cached --check` and `git status --short` are reviewed.
- ☐ Documentation/manual is included in the feature branch with meaningful Git commit(s).
- ☐ Branch is published only when ready; create a PR targeting `main`.
- ☐ Review PR diff and status; merge when verified, then update local `main`.

Appendix A. Reference interfaces (actual basic design)

A.1 `tokenizer.hpp`

```cpp
#pragma once
#include <cstddef>
#include <string>
#include <vector>

enum class TokenType {
    Keyword,
    Identifier,
    Number,
    StringLiteral,
    CharLiteral,
    Operator,
    Punctuation,
    Unknown
};

struct Token {
    TokenType type;
    std::string lexeme;
    std::size_t line;
    std::size_t column;
};

std::vector<Token> tokenize(const std::string& source);
```

A.2 `preprocessor.hpp`

```cpp
#ifndef CLONESCOPE_PREPROCESSOR_HPP
#define CLONESCOPE_PREPROCESSOR_HPP

#include <string>

// Remove comments while preserving code and line positions.
std::string removeComments(const std::string &source);

#endif
```

A.3 `application.hpp`

```
#pragma once

// Start and coordinate the CloneScope application.
// Returns an exit status to the operating system.
int runCloneScope();
```

Appendix B. Tokenization quick-reference tables

B.1 Location update rules

Region	Movement	Location handling
Identifier or number	One char at a time	Increment `index` and `column`
Two-char operator	Two chars together	`index += 2`, `column += 2`
Quoted escape pair	Two chars together	`index += 2`, `column += 2`
Whitespace other than LF	One char	Increment column, emit no token
LF outside quote scanner	One char	Increment line, set column 1
Unsupported non-whitespace	One char	Emit `Unknown`, then increment position

B.2 Supported single-/two-character operator declaration

```
static const std::string singleCharOperators = "+-*/%&=!<>|^~.";
static const std::unordered_set<std::string> twoCharOperators = {
    "==", "++", "--", "+=", "-=", "*=", "/=",
    "%=", "&=", "|=", "^=", "!=", "<=", ">=",
    "&&", "||", "::", "->"
};
```

Appendix C. Further self-study: reading order

If any C++ syntax remains unfamiliar, revise in this order:

1. Strings, character literals and escape sequences (`\n`, `\\`, `\"`, `\'`).
2. Arrays/vectors and zero-based indexing.
3. `struct`, `enum class`, scope resolution (`::`).
4. `const` references, return types, function declarations/definitions.
5. `while`, `for`, `if/else if`, `break`, `continue`.
6. `<cctype>`, `unsigned char`, `static_cast`.
7. File streams, string streams and `std::filesystem`.
8. Hash sets and membership queries.
9. State machines, test assertions and regression debugging.
10. Git staging, meaningful commits, pull requests and review.

Closing reminder: A strong SPL viva answer links each design decision to a need: accurate token boundaries, preserved source locations, maintainable modules, reproducible tests, transparent scope limits and explainable later evidence. That is the purpose of this manual.